

Verification Continuum™

AE-ware Application Note FSM Livelock/Deadlock and Fairness Constraint

SYNOPSYS®

Copyright Notice and Proprietary Information

©2021 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <http://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

AE-ware Application Note for FSM Livelock/Deadlock and Fairness Constraint

This application note guides the users how to use this AE-ware to execute FSM lock check and solve false failures with fairness constraints.

For product support, see <https://www.synopsys.com/support.html>.

Introduction

- This AE-ware solution is for the generation of FSM Livelock and Deadlock checks and Fairness Constraints.
 - The checks are generated in the form of SVA and can be checked in either AEP or FPV App.
 - Deadlock is to check for infinite wait in a single state. If FSMs enter a given state and stay in the same state which can't transition to other states.
 - Livelock is to check if a FSM get stuck between multiple states.
 - Fairness constraint is to make sure the FSM COI input could be activated and would not be stuck in a specific value to get FSM lock.

Prerequisites

1. VC Formal version needs to be 2020.12-SP2-2, 2021.09 or higher versions.
2. AE-ware location
\$VC_STATIC_HOME/auxx/ctg/tcl/library/proc/aep_generation.tbc

Recommended Steps

1. Enable the proc first in VCF console or tcl as follows:

```
% source < should be $VC_STATIC_HOME of VCF install  
dir>/auxx/ctg/tcl/library/proc/aep_generation.tbc
```
2. Add FSM lock checker with this aep_generate command:
 - Usage: aep_generate # generate and enable livelock, deadlock and unionlock assertions for each FSM state in the design.
 - [-fsm_deadlock] (Default: 1. Set to 0 to disable deadlock assertions)
 - [-fsm_livelock] (Default: 0. Set to 1 to enable livelock assertions)
 - [-fsm_unionlock] (Default: 0. Set to 1 to generate single assertion for deadlock and livelock checking)
 - [-fsm_fairness] (Generate FSM fairness assumptions for each FSM state, enable this option would disable fsm_*lock generation)
 - For detail, please go to the pages of aep_generate -fsm_fairness
3. Add FSM fairness constraint with this aep_generate -fsm_fairness command
 - Usage: aep_generate -fsm_fairness # Generate all FSM COI input fairness constraints. Output tcl is aep_fsm_fairness.tcl
 - [-input] (Generate fairness constraint for the specified inputs).
 - [-ignore_width] (Inputs greater than this width will be ignored. Default is 2)
 - [-list_coi] (List all FSM COI inputs)
4. The following flowchart explains the methodology of how to run the FSM lock check with this AE-ware:

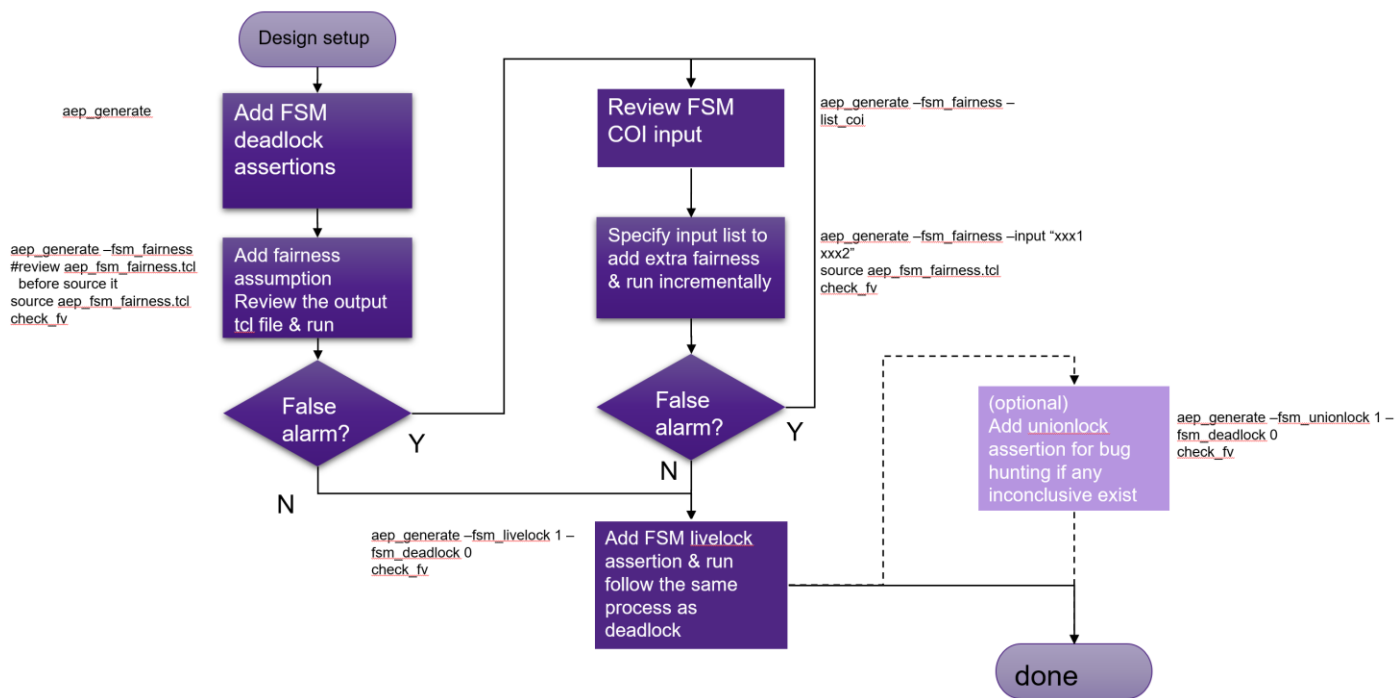


Figure 1: FSM deadlock/livelock + fairness constraint methodology

Example

To generate lock checker, please follow the example below

##generate deadlock assertions for all FSM states

% aep_generate

generate unionlock assertions for each FSM state

% aep_generate -fsm_unionlock 1 -fsm_deadlock 0

generate and enable only livelock assertions for each FSM state

% aep_generate -fsm_livelock 1 -fsm_deadlock 0

generate deadlock and livelock assertions for each FSM state

% aep_generate -fsm_livelock 1

	status	depth	name
1			add_fsm_lock_traffic.first.state_out_RESET_deadlock
2			add_fsm_lock_traffic.first.state_out_SG_deadlock
3			add_fsm_lock_traffic.first.state_out_SR0_deadlock
4			add_fsm_lock_traffic.first.state_out_SR1_deadlock
5			add_fsm_lock_traffic.first.state_out_SY_deadlock

Figure 2: Added FSM deadlock properties by aep_generate example, FSM has 5 states: RESET, SG, SR0, SR1, SY

To generate fairness constraint, please follow the example below

Show FSM COI input to review

```
% aep_generate -fsm_fairness -list_coi
```

Generate fairness assumes for inputs up to 2 bits

```
% aep_generate -fsm_fairness -ignore_width 3
```

Generate fairness assumes with inputs specified by users

```
% aep_generate -fsm_fairness -input "in1 in2 in3"
```

```
fvasume add_fsm_constraint_in_0 -expr {@ (posedge clk) s_eventually in == 0}
fvasume add_fsm_constraint_in_1 -expr {@ (posedge clk) s_eventually in == 1}
fvasume add_fsm_constraint_in_2 -expr {@ (posedge clk) s_eventually in == 2}
fvasume add_fsm_constraint_in_3 -expr {@ (posedge clk) s_eventually in == 3}
```

Figure 3: This generated fairness constraint is a example for a 2-bit input port "in"

Known Limitations

While validated on internal and customer designs, this AE-ware has not undergone the same level of product validation scrutiny as production VC Formal features.

Conclusion

This AE-ware is available as an immediate solution to cover the requirement for FSM lock check.

Note: Complete support within VC Formal is being developed and will be available soon.